



UNIVERSITÉ ÉVRY
PARIS-SACLAY



université
PARIS-SACLAY

UNIVERSITÉ D'ÉVRY - PARIS SACLAY
Faculté des Sciences et de l'Ingénierie

COMPTE RENDU

Connectivité et traitement de données

Socket Messaging

Réalisé par :

Benazzouz Kaddour

Master 2 ISC – Robotique Industrielle
UNIVERSITÉ D'ÉVRY - PARIS SACLAY

Responsable de l'UE :

M.Lamri Nehaoua

Année universitaire 2025–2026

Paris Saclay, le November 26, 2025

Table des matières

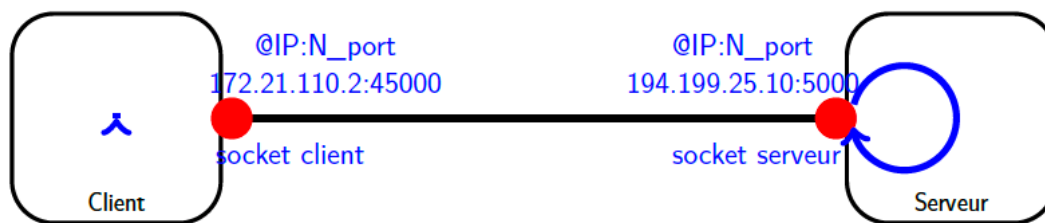
Partie TD	3
Concept Socket	3
1. Pourquoi le protocole IP ne suffit pas à garantir un transfert fiable des données	3
2. Comment la couche transport améliore-t-elle la fiabilité des échanges	3
3. Citez les deux protocoles principaux utilisés à cette couche et leur classification	3
a) TCP (Transmission Control Protocol) : Orienté Connexion	3
b) UDP (User Datagram Protocol) : Non Orienté Connexion	4
4. Exemples d'applications typiques utilisant les protocoles de la couche transport	4
5. Protocoles applicatifs industriels utilisant les protocoles de la couche transport	4
6. Compréhension du concept de socket	5
a) Qu'est-ce qu'une socket ?	5
b) Fonction d'un port :	5
c) Multiplexage :	5
d) Nombre et classification des ports :	5
7. L'ouverture d'une connexion TCP (Three-way handshake)	5
a) Étapes :	5
b) Rôle des numéros de séquence et d'acquittement :	5
8. Échanges TCP et formats PDU TCP/UDP	6
a) Échanges TCP sans superposition	6
b) Échanges TCP avec superposition	6
9. Recherche et formats PDU TCP/UDP + Rôle des champs	6
a) Format PDU TCP (Segment TCP)	6
b) Format PDU UDP (Datagramme UDP)	7
10. Analyse de l'automate TCP	8
a) Liste des états possibles du protocole TCP et leur signification	8
b) Signification des mots-clés associés aux transitions	8
c) En quoi les transitions menant à l'état ESTABLISHED reflètent-elles exactement le three-way handshake	8
d) Proposer deux automates TCP simplifiés (client/serveur)	9
e) Discussion sur les états WAIT et CLOSE_WAIT	9
11. Notion bind, listen et socket éphémère	10
a) Liaison d'une socket avec bind	10
b) Rôle de la socket serveur (avec listen())	10
c) Les deux sockets impliquées dans la communication finale	10
API de programmation de sockets	11
12. Étude de l'API WinSock (C sous Windows)	11
13. API BSD Unix (sous Unix BSD / POSIX)	11
14. API Python (module socket)	12
15. Implémentation d'une communication Client/serveur	12
Socket Messaging sur Robot	16
16. Tag réseau dans la configuration FANUC	16
17. À quoi correspond un tag côté serveur ? côté client ?	16
18. Analogie avec la notion de socket dans WinSock ou BSD	16
19. Fonctions principales pour implémenter une communication socket	17
20. Gestion de la socket en KAREL	17
21. Étapes obligatoires pour établir une communication socket dans un programme KAREL	18
22. Exemple simplifié de programme KAREL	18
23. Analyse et décapsulation manuelle d'une trame Ethernet et d'un paquet IPv4 avec explication du bourrage	19

24.Capture et analyse d'une requête et d'une réponse HTTP avec reconstruction du segment TCP	21
Partie TP	22
Communication entre Karel et Python via Sockets	22
1. Introduction	22
2. Environnement Utilisé	22
3. Configuration de la Communication dans RoboGuide	22
4. Programme KAREL	25
5. Script Python	26
6. Démonstration du Fonctionnement	27
7. Conclusion	28
8. Contrôle du Robot via Python et KAREL	28

Partie TD

Dans le cadre de l'intégration industrielle moderne, il devient essentiel de permettre à un robot de communiquer avec un système de supervision distant. Cette communication s'effectue typiquement en mode client/serveur TCP/IP, à travers une messagerie par sockets. Le robot, via un programme spécifique, joue le rôle de client à savoir initier une connexion vers un PC distant. De son côté, le PC distant exécute une application (écrite en Python, LabVIEW, Node.js, etc.) qui agit en serveur socket et écoute en permanence les requêtes du robot.

Le mécanisme Socket repose sur une pile réseau TCP/IP classique et fonctionne indépendamment du support de communication (Ethernet, PPP série, VPN, etc.). Il est compatible avec la plupart des langages et outils standards, sans recourir à des protocoles industriels propriétaires. Cela en fait un levier puissant pour intégrer les robots dans une architecture IIoT évolutive, avec supervision à distance, journalisation centralisée, ou même interfaçage cloud (via MQTT, par exemple).



figureSchéma de communication client/serveur via sockets TCP/IP.

Concept Socket

1. Pourquoi le protocole IP ne suffit-il pas à garantir un transfert fiable des données

Le protocole IP (Internet Protocol) permet le routage des paquets de données, mais il ne garantit pas la fiabilité de leur transfert. En effet, il n'assure pas la correction des erreurs, la détection de pertes de paquets, ni leur retransmission. C'est pourquoi une couche de transport, comme TCP (Transmission Control Protocol), est nécessaire pour assurer un transfert fiable des données (en assurant l'ordre des paquets et leur retransmission si nécessaire).

2. Comment la couche transport améliore-t-elle la fiabilité des échanges

La couche de transport, par exemple TCP, améliore la fiabilité des échanges en assurant des fonctionnalités telles que :

- **Le contrôle de flux** : elle régule la quantité de données envoyées pour ne pas saturer le récepteur.
- **La gestion des erreurs** : elle assure la détection et la retransmission des paquets perdus.
- **La gestion de l'ordre des paquets** : elle garantit que les paquets arrivent dans le bon ordre.
- **L'établissement et la terminaison de la connexion** : via le mécanisme de "handshake" dans TCP.

3. Citez les deux protocoles principaux utilisés à cette couche et leur classification

Les deux protocoles principaux utilisés à cette couche sont :

a) TCP (Transmission Control Protocol) : Orienté Connexion

C'est un protocole orienté connexion, car il établit une connexion, garantit l'ordre et la fiabilité de l'envoi des paquets. TCP est un protocole orienté connexion, ce qui signifie qu'une connexion doit être établie entre le client et le serveur avant tout échange de données. Voici les caractéristiques et le fonctionnement détaillé :

- **Établissement de la connexion** : Avant de commencer à envoyer des données, une séquence d'établissement de connexion doit être effectuée (ce que l'on appelle le Three-Way Handshake). Cette étape permet de s'assurer que les deux parties (client et serveur) sont prêtes à échanger des données.

- Le client envoie un message SYN (synchronize) pour initier la connexion.
- Le serveur répond avec un message SYN-ACK (synchronize-acknowledge) pour accepter la connexion.
- Le client répond par un message ACK (acknowledge) pour confirmer l'établissement de la connexion.
- **Fiabilité et Contrôle d'Erreur** : TCP garantit que les paquets arrivent dans le bon ordre, qu'ils ne sont pas perdus, et qu'ils sont bien reçus. Si un paquet est perdu en cours de route, il sera renvoyé par le serveur ou le client. Cette fiabilité est obtenue grâce à un mécanisme de numéros de séquence et accusés de réception (ACK) qui permet de suivre l'état des paquets.
- **Contrôle de flux et congestion** : TCP utilise des mécanismes comme le contrôle de flux et le contrôle de congestion pour ajuster la vitesse d'envoi des paquets, ce qui aide à éviter la surcharge du récepteur ou du réseau.
- **Fermeture de la connexion** : Une fois la communication terminée, la connexion TCP doit être fermée proprement par un processus de terminaison (grâce à un échange de paquets FIN).

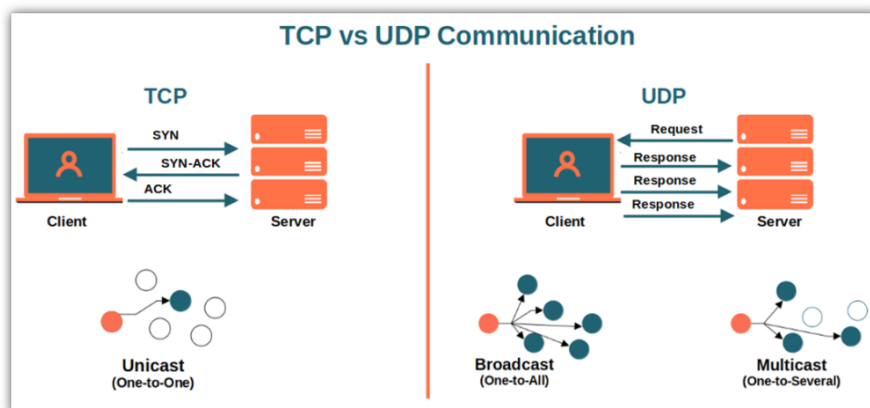
b) UDP (User Datagram Protocol) : Non Orienté Connexion

UDP, en revanche, est un protocole non orienté connexion, ce qui signifie qu'il n'y a pas de processus d'établissement de connexion entre le client et le serveur avant l'échange de données. Voici ses caractéristiques et fonctionnement :

- **Absence d'établissement de connexion** : Avec UDP, une fois qu'un message est envoyé, il n'y a pas de confirmation ou d'échange préalable pour garantir que le récepteur est prêt à recevoir les données. Il n'y a pas de "handshake". Le client envoie un paquet (datagramme) et le serveur reçoit ce paquet, sans garantir que la connexion existe au préalable.
- **Pas de garantie de livraison** : UDP ne garantit pas la livraison des paquets. Si un paquet est perdu ou corrompu, il ne sera pas renvoyé. De plus, il ne garantit pas l'ordre d'arrivée des paquets (les paquets peuvent arriver dans un ordre différent de celui dans lequel ils ont été envoyés).
- **Pas de contrôle de congestion ou de flux** : UDP n'intègre pas de mécanismes de contrôle de congestion. Cela peut rendre UDP plus rapide dans des situations où la vitesse est prioritaire et où une certaine perte de données est acceptable, mais cela peut également entraîner des risques de perte de données sans qu'il y ait de retransmission automatique.

4. Exemples d'applications typiques utilisant les protocoles de la couche transport

- **TCP** : navigation web (HTTP/HTTPS), transfert de fichiers (FTP), envoi d'e-mails (SMTP)
- **UDP** : jeux en ligne, diffusion en continu (streaming audio et vidéo)



5. Protocoles applicatifs industriels utilisant les protocoles de la couche transport

Des protocoles comme Modbus/TCP, OPC UA (Unified Architecture), et MQTT sont souvent utilisés dans les systèmes industriels pour la communication via la couche transport (TCP ou UDP).

6. Compréhension du concept de socket

- a) Qu'est-ce qu'une socket ?

Une socket est un point de communication entre deux processus qui permet d'échanger des données via un réseau, en utilisant un couple (adresse IP, numéro de port).

- b) Fonction d'un port :

Le port permet de différencier les différentes connexions et applications sur une même machine, en attribuant un numéro spécifique à chaque service ou application qui utilise la communication réseau.

- c) Multiplexage :

Les ports permettent à plusieurs applications de partager la même interface réseau en utilisant des numéros différents. Ainsi, différentes applications peuvent envoyer et recevoir des données via la même adresse IP, mais en utilisant des ports différents.

- d) Nombre et classification des ports :

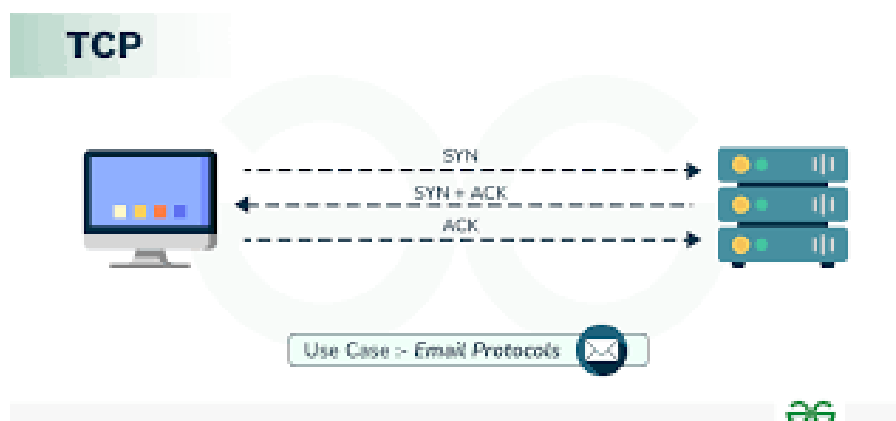
Il existe 65 536 ports possibles (numérotés de 0 à 65535). Ces ports sont classés en trois catégories :

- Well-known ports (0-1023) : utilisés par des services communs comme HTTP (80), HTTPS (443), FTP (21)
- Registered ports (1024-49151) : utilisés par des applications moins courantes mais enregistrées
- Dynamic ports (49152-65535) : utilisés par les clients pour établir des connexions temporaires

7. L'ouverture d'une connexion TCP (Three-way handshake)

- a) Étapes :

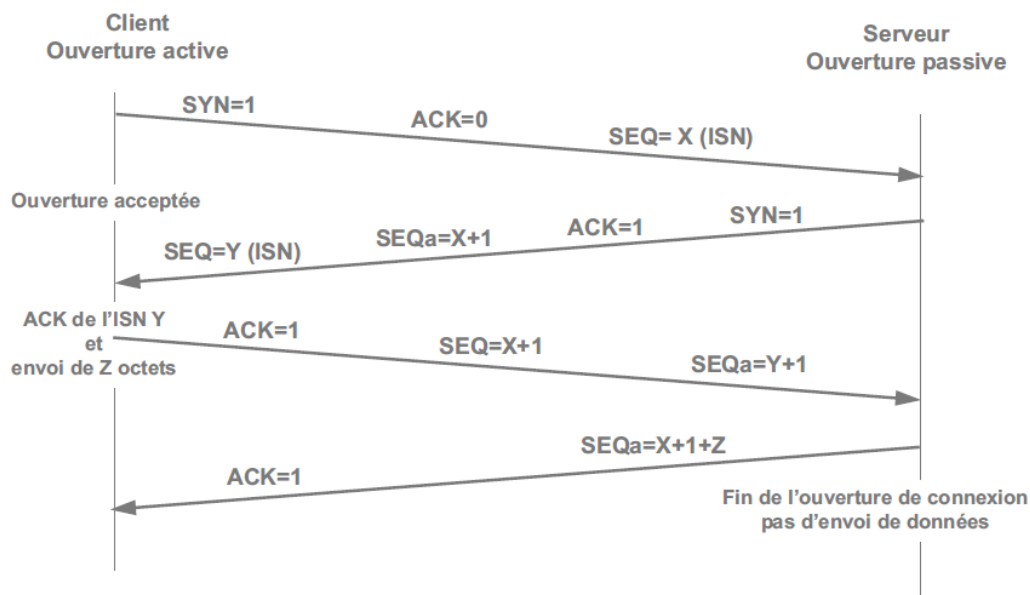
- SYN : Le client envoie un paquet SYN pour initier une connexion
- SYN-ACK : Le serveur répond par un paquet SYN-ACK pour accepter
- ACK : Le client renvoie un paquet ACK pour confirmer



- b) Rôle des numéros de séquence et d'acquittement :

Ils permettent de suivre l'ordre des paquets échangés et de garantir qu'ils sont reçus correctement. Le numéro de séquence identifie le premier octet d'un paquet, et le numéro d'acquittement indique que des octets ont été correctement reçus.

- c) **Pourquoi nécessaire** : Cette phase est cruciale pour établir une connexion fiable entre les deux parties, synchroniser leurs séquences de numéros et s'assurer qu'elles sont prêtes à échanger des données de manière ordonnée et fiable.



8. Échanges TCP et formats PDU TCP/UDP

a) Échanges TCP sans superposition

Chaque segment de données est immédiatement suivi d'un acquittement (ACK). Exemple :

- Le client envoie un mot (exemple : "kaddour") sous forme de segment TCP au serveur.
- Le serveur accuse réception en envoyant un ACK.
- Le serveur envoie ensuite un message d'écho (exemple : "écho").
- Le client accuse réception en envoyant un ACK.

b) Échanges TCP avec superposition

Dans un échange avec superposition de messages, les acquittements sont retardés, et les données sont envoyées dans le même segment que l'ACK. Cela donne : Le client envoie un segment TCP contenant « kaddour ». Le serveur ne répond pas immédiatement avec un ACK. Il attend de pouvoir envoyer une donnée en retour (ici "écho"). Ensuite, le serveur envoie un segment contenant à la fois les données ("écho") et l'ACK pour le message reçu. Le client fait de même : il envoie un segment TCP contenant "kaddour" et l'ACK pour "écho".

9. Recherche et formats PDU TCP/UDP + Rôle des champs

a) Format PDU TCP (Segment TCP)

Port source	Port destination
Numéro de séquence	
Numéro d'acquittement (ACK)	
Offset	Reserved
Flags (SYN,ACK..)	
Fenêtre	
Checksum	Urgent Pointer
Options (facultatif)	
Données (payload)	

+-----+

Rôle des champs :

- **Ports source/destination** : identifient les applications (services).
- **Numéro de séquence** : ordre des données envoyées.
- **Numéro d'acquittement** : confirmation de réception.
- **Flags** : contrôle la session (SYN, ACK, FIN...).
- **Fenêtre** : gestion du flux (taille de la fenêtre de réception).
- **Checksum** : vérification d'erreur.
- **Données** : contenu réel transmis.

Philosophie de TCP :

- Orienté connexion.
- Fiable (acquittement, ordre, retransmission).
- Contrôle de flux et de congestion.

b)Format PDU UDP (Datagramme UDP)

```
+-----+
| Port source      | Port destination |
+-----+
| Longueur totale  | Checksum         |
+-----+
| Données (payload) |
+-----+
```

Rôle des champs :

- **Ports source/destination** : identifient les services.
- **Longueur** : taille totale du datagramme.
- **Checksum** : vérification simple (parfois optionnelle).
- **Données** : contenu envoyé.

Philosophie de UDP :

- Non orienté connexion.
- Pas de garantie de livraison.
- Très rapide, mais sans vérification d'erreur ou de réception.

Protocole	Connexion	Fiabilité	Vitesse	Utilisation typique
TCP	Oui	✓ Fiable (ACK, retransmission)	Moins rapide	Web, FTP, email
UDP	Non	✗ Non fiable	Très rapide	Streaming, jeux, VoIP

10. Analyse de l'automate TCP

a) Liste des états possibles du protocole TCP et leur signification

État	Signification
CLOSED	Aucune connexion n'est active ou en cours.
LISTEN	Serveur en attente d'une demande de connexion (passive open).
SYN_SENT	Le client a envoyé un SYN, attente de réponse du serveur (active open).
SYN_RCVD	Le serveur a reçu un SYN et a envoyé un SYN-ACK, attente de l'ACK du client.
ESTABLISHED	Connexion complètement établie. Les deux côtés peuvent échanger des données.
FIN_WAIT_1	Une des deux parties a envoyé un FIN pour fermer la connexion.
FIN_WAIT_2	FIN a été acquittée, en attente de FIN de l'autre côté.
CLOSING	Les deux parties ferment en même temps (envoi et réception de FIN).
TIME_WAIT	Attente pour s'assurer que l'autre partie a bien reçu l'ACK du FIN (éviter les doublons).
CLOSE_WAIT	Une FIN a été reçue, mais l'application locale ne l'a pas encore confirmée.
LAST_ACK	L'application a envoyé un FIN après avoir reçu un FIN, en attente de l'ACK.

b) Signification des mots-clés associés aux transitions

Transition	Signification
send SYN	Le client initie une demande de connexion (SYN).
receive SYN	Réception d'une demande de connexion.
send ACK	Envoi d'un acquittement.
receive ACK	Réception d'un acquittement.
send FIN	Envoi d'une demande de fermeture de connexion.
receive FIN	Réception d'une demande de fermeture.
delete TCB	Libération des ressources (Transmission Control Block).
CLOSE	Demande locale de fermeture de la connexion.

c) En quoi les transitions menant à l'état **ESTABLISHED** reflètent-elles exactement le three-way handshake

Le three-way handshake TCP (établissement de connexion en 3 étapes) est représenté par ces transitions :

- Client (état **CLOSED**) → **send SYN** → **SYN_SENT**
- Serveur (état **LISTEN**) → **receive SYN** + **send SYN, ACK** → **SYN_RCVD**
- Client → **receive SYN, ACK** + **send ACK** → **ESTABLISHED**
- Serveur → **receive ACK** → **ESTABLISHED**

Donc pour récapituler :

- Le client initie la connexion.
- Le serveur répond avec SYN+ACK.
- Le client finalise avec un ACK.

→ L'état **ESTABLISHED** est atteint des deux côtés après les 3 étapes.

d) Proposer deux automates TCP simplifiés (client/serveur)**Automate simplifié – Client :**

1. CLOSED
2. → (active open, send SYN) → SYN_SENT
3. → (receive SYN+ACK, send ACK) → ESTABLISHED
4. → (send FIN) → FIN_WAIT_1
5. → (receive ACK) → FIN_WAIT_2
6. → (receive FIN, send ACK) → TIME_WAIT
7. → (timeout) → CLOSED

Automate simplifié – Serveur :

1. CLOSED
2. → (passive open) → LISTEN
3. → (receive SYN, send SYN+ACK) → SYN_RCVD
4. → (receive ACK) → ESTABLISHED
5. → (receive FIN, send ACK) → CLOSE_WAIT
6. → (send FIN) → LAST_ACK
7. → (receive ACK) → CLOSED

e) Discussion sur les états WAIT et CLOSE_WAIT**À quoi correspondent ces états en termes de blocage :**

- Les états WAIT (comme TIME_WAIT, FIN_WAIT_1/2) sont souvent gérés au niveau du noyau (kernel) et non bloquants pour l'application directement.
- CLOSE_WAIT signifie que le programme local n'a pas encore fermé la connexion (il attend probablement une commande `close()` de l'application).

Un programme dans l'état LISTEN ou WAIT est-il actif ou bloqué :

- LISTEN : actif (attend passivement des connexions entrantes).
- WAIT (ex. TIME_WAIT) : pas bloqué, mais le système garde la ressource occupée.

Conséquences sur la gestion des ressources : Si trop de connexions restent en TIME_WAIT, cela peut :

- Saturer la table de connexions TCP (épuisement des ports).
- Ralentir les nouvelles connexions.
- Exiger d'augmenter les délais de recyclage de ports.

Pourquoi le timeout est-il nécessaire après l'état TIME_WAIT : Permet d'attendre le double du temps maximal de vie d'un segment (2MSL) pour :

- S'assurer que l'ACK du FIN a bien été reçu par l'autre partie.
- Empêcher les doublons d'anciens segments d'interférer avec de nouvelles connexions utilisant le même couple IP:port.

11. Notion bind, listen et socket éphémère

a) Liaison d'une socket avec bind

Rôle de bind() :

La fonction `bind()` sert à associer une socket à une adresse IP locale et un port spécifique sur la machine. Cela permet au système d'exploitation de savoir que cette socket sera utilisée pour recevoir les données envoyées à cette adresse et ce port.

Est-ce que le client doit toujours faire un bind() explicite ?

Non, le client n'a généralement pas besoin d'appeler `bind()` explicitement.

Quand un client crée une socket et initie une connexion vers un serveur, le système d'exploitation lui attribue automatiquement une adresse IP locale et un port éphémère (non réservé) pour la communication.

Que se passe-t-il si plusieurs sockets essaient de se lier au même port ?

Par défaut, deux sockets ne peuvent pas se lier au même port sur la même adresse IP, sinon il y aura une erreur (`EADDRINUSE`).

Cependant, il est possible d'autoriser plusieurs sockets à partager un port via certaines options comme `SO_REUSEADDR`, mais cela a des règles strictes et est surtout utilisé côté serveur.

b) Rôle de la socket serveur (avec listen())

Rôle de la socket serveur avec listen() :

La socket serveur est mise en mode écoute avec `listen()`. Cela signifie qu'elle attend des demandes de connexion entrantes. Elle ne communique pas directement avec les clients, elle accepte les connexions.

File d'attente dans le schéma :

La file d'attente représente les demandes de connexion en attente d'acceptation (via `accept()`).

Quand plusieurs clients essaient de se connecter en même temps, leurs requêtes sont mises en queue.

Pourquoi cette socket ne sert pas directement à communiquer ?

La socket en écoute ne transporte pas directement les données échangées.

Elle sert uniquement à gérer les demandes de connexion entrantes.

Différence entre la socket initiale (écoute) et la socket créée dynamiquement (socket éphémère)

:

- La socket initiale est la socket d'écoute qui reste disponible pour accepter des connexions.
- La socket éphémère est créée par le serveur lors de l'appel à `accept()` pour gérer la communication avec un client spécifique.

Pourquoi cette socket est appelée « socket éphémère » ?

Parce qu'elle existe uniquement pendant la durée de la connexion avec ce client. Une fois la connexion terminée, elle est fermée et détruite.

c) Les deux sockets impliquées dans la communication finale

Côté client :

Le client utilise une socket (avec un port éphémère attribué automatiquement) pour envoyer et recevoir des données vers/depuis le serveur.

Côté serveur :

Le serveur utilise une socket éphémère, créée dynamiquement après `accept()`, pour communiquer avec ce client précis.

Que se passe-t-il si plusieurs clients se connectent au même serveur ?

Le serveur crée une socket éphémère distincte pour chaque client connecté.

Ainsi, chaque client a sa propre connexion indépendante.

Le serveur a-t-il besoin d'un thread ou processus séparé pour chaque socket éphémère ?

Pas nécessairement, mais c'est courant en pratique pour gérer plusieurs connexions simultanément :

- On peut utiliser des threads/processus séparés pour chaque socket éphémère pour gérer la concurrence facilement.
- Sinon, on peut gérer plusieurs sockets avec des mécanismes comme `select()`, `poll()` ou `epoll()` dans un seul thread.

API de programmation de sockets

12. Étude de l'API WinSock (C sous Windows)

Principales fonctions nécessaires :

- `socket()` : Crée une nouvelle socket.
- `bind()` : Associe l'adresse IP et le port à une socket.
- `listen()` : Permet à la socket d'écouter les connexions entrantes.
- `accept()` : Accepte une connexion entrante.
- `connect()` : Établit une connexion à un autre hôte via une socket.
- `send()` : Envoie des données à travers la socket.
- `recv()` : Reçoit des données via la socket.

Prototypes des fonctions :

```
int socket(int domain, int type, int protocol)
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)
int listen(int sockfd, int backlog)
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen)
int send(int sockfd, const void *buf, size_t len, int flags)
int recv(int sockfd, void *buf, size_t len, int flags)
```

Structures d'adressage :

- `struct sockaddr` : Représente une adresse générique.
- `struct sockaddr_in` : Représente une adresse IPv4.
- `struct sockaddr_in6` : Représente une adresse IPv6.

Fonctions utilitaires :

- `inet_addr()` : Convertit une adresse IPv4 en format numérique.
- `inet_ntoa()` : Convertit une adresse IPv4 sous forme numérique en une chaîne de caractères.

13. API BSD Unix (sous Unix BSD / POSIX)

Les fonctions principales sous BSD Unix sont similaires à celles de WinSock :

- `socket()`
- `bind()`
- `listen()`
- `accept()`
- `connect()`
- `send()`
- `recv()`

Structure `sockaddr_in` : similaire à WinSock avec quelques différences subtiles dans la gestion des familles d'adresses et des types de socket.

14 API Python (module socket)

Les fonctions de socket en Python sont abstraites sous forme d'objets :

- `socket.socket()`
- `socket.bind()`
- `socket.listen()`
- `socket.accept()`
- `socket.connect()`
- `socket.send()`
- `socket.recv()`

Objets retournés :

- L'objet socket est retourné pour établir une connexion.
- Les adresses et les ports sont souvent représentés sous forme de tuples, par exemple : `("localhost", 8080)`.

Représentation des adresses IP et ports en Python :

- Adresses IP : chaînes de caractères (ex. `"127.0.0.1"` pour IPv4)
- Ports : entiers (ex. `port = 5000`)

15. Implémentation d'une communication Client/serveur

15.1 Objectif

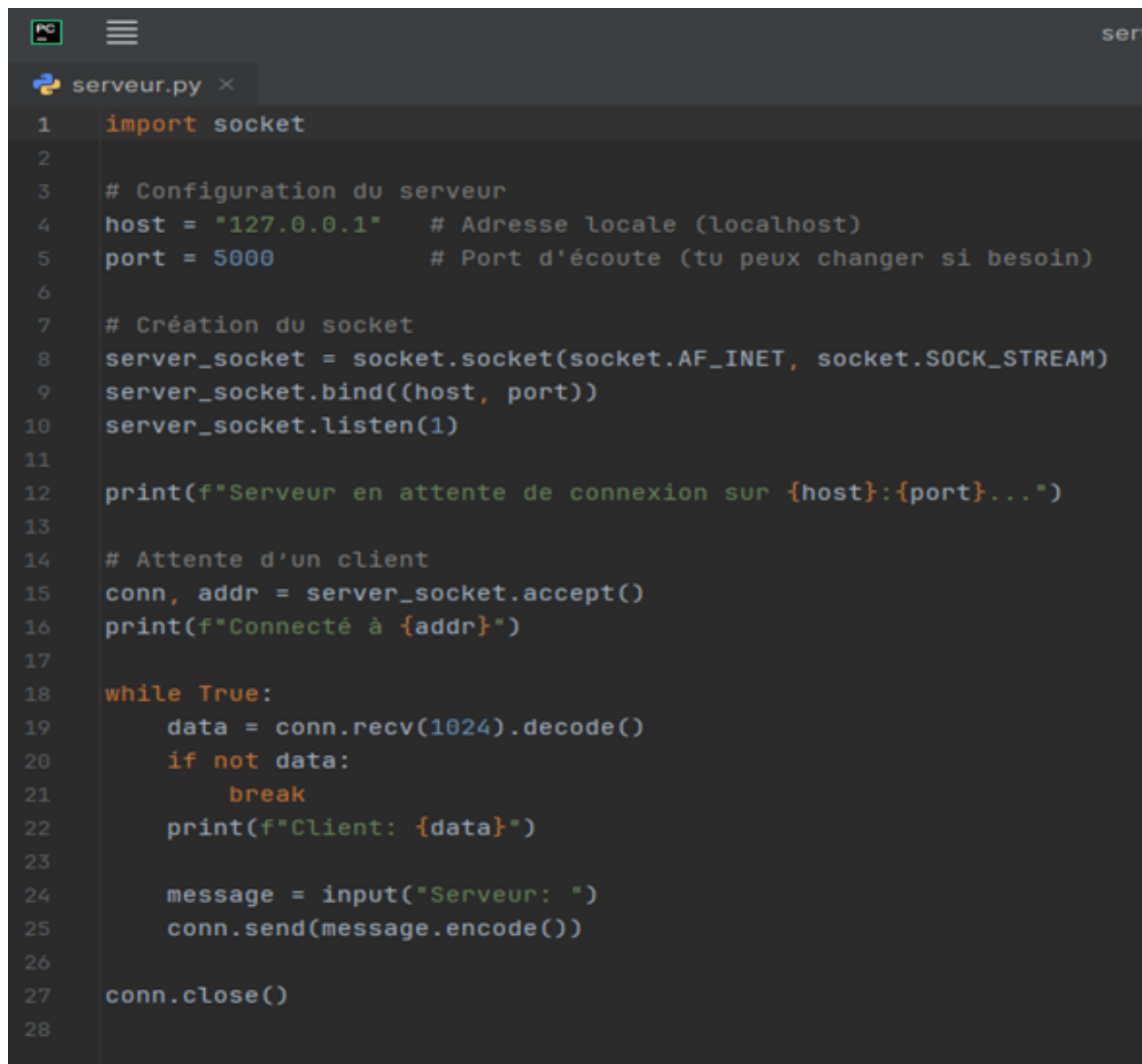
L'objectif de ce travail est de réaliser une communication réseau simple entre un serveur et un client en langage python.

15.2 Étapes de réalisation

1. Création des projets :
 - Projet Serveur
 - Projet Client
2. L'adresse 127.0.0.1 (localhost) a été utilisée afin d'établir la communication en local, sans nécessiter de réseau externe.
3. Procédure d'exécution :
 - D'abord, le serveur est exécuté dans une première fenêtre de terminal (`cmd.exe`).
 - Ensuite, le client est exécuté et se connecte via l'adresse IP locale et le port spécifié.
 - Une fois la connexion établie, un échange de messages a lieu.

a) Côté Serveur : Programme Python

Voici le programme du serveur développé en langage python. Il se lie à l'adresse **127.0.0.1** et au port **5000**, puis se met en attente d'une connexion client. Une fois le client connecté, le serveur reçoit un message et envoie une réponse.

Programme Serveur :A screenshot of a code editor window titled 'serveur.py'. The code is written in Python and implements a simple server. It imports the 'socket' module, configures the host to '127.0.0.1' and port to '5000', creates a server socket, binds it, and listens for connections. Once a connection is accepted, it prints the client address, receives data, prints it, and then sends a response 'Serveur: ' back to the client. The code ends with a loop that continues until the connection is closed.

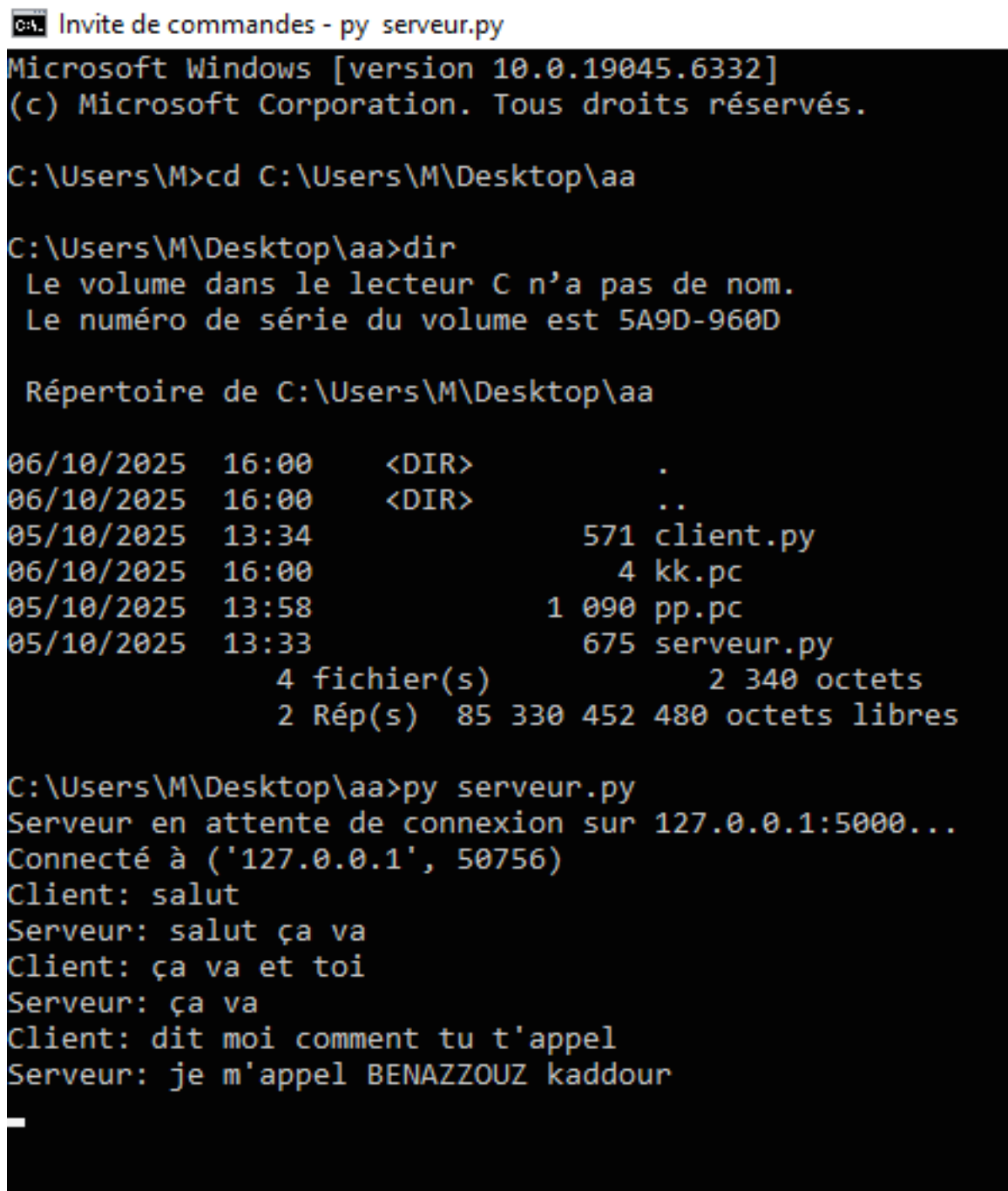
```
1 import socket
2
3 # Configuration du serveur
4 host = "127.0.0.1" # Adresse locale (localhost)
5 port = 5000        # Port d'écoute (tu peux changer si besoin)
6
7 # Création du socket
8 server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9 server_socket.bind((host, port))
10 server_socket.listen(1)
11
12 print(f"Serveur en attente de connexion sur {host}:{port}...")
13
14 # Attente d'un client
15 conn, addr = server_socket.accept()
16 print(f"Connecté à {addr}")
17
18 while True:
19     data = conn.recv(1024).decode()
20     if not data:
21         break
22     print(f"Client: {data}")
23
24     message = input("Serveur: ")
25     conn.send(message.encode())
26
27 conn.close()
28
```

Exécution côté CMD :

1. Ouvrir une première fenêtre de terminal (`cmd.exe`).
2. Se placer dans le dossier contenant le fichier `serveur.py`.
3. Lancer le programme avec la commande :

```
python serveur.py
```

4. Une fois le serveur lancé et en attente, ouvrir une deuxième fenêtre de terminal pour exécuter le client.



```
C:\> Invite de commandes - py serveur.py
Microsoft Windows [version 10.0.19045.6332]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\M>cd C:\Users\M\Desktop\aa

C:\Users\M\Desktop\aa>dir
Le volume dans le lecteur C n'a pas de nom.
Le numéro de série du volume est 5A9D-960D

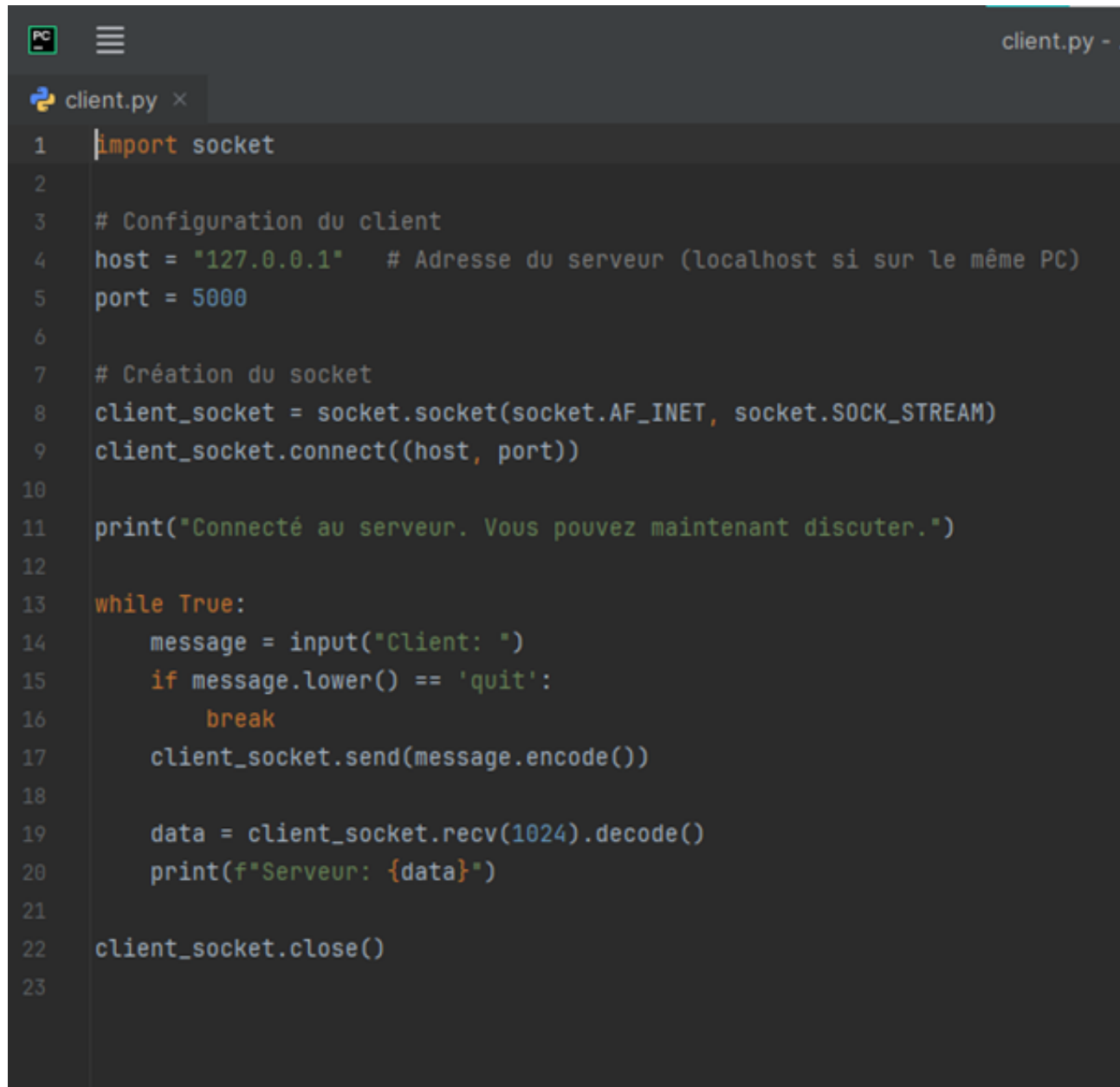
Répertoire de C:\Users\M\Desktop\aa

06/10/2025  16:00    <DIR>          .
06/10/2025  16:00    <DIR>          ..
05/10/2025  13:34             571 client.py
06/10/2025  16:00              4 kk.pc
05/10/2025  13:58           1 090 pp.pc
05/10/2025  13:33           675 serveur.py
               4 fichier(s)             2 340 octets
               2 Rép(s)  85 330 452 480 octets libres

C:\Users\M\Desktop\aa>py serveur.py
Serveur en attente de connexion sur 127.0.0.1:5000...
Connecté à ('127.0.0.1', 50756)
Client: salut
Serveur: salut ça va
Client: ça va et toi
Serveur: ça va
Client: dit moi comment tu t'appel
Serveur: je m'appel BENAZZOUZ kaddour
```

b) Côté Client : Programme Python

Voici le programme du client développé en langage python. Crée un socket, puis se connecte au serveur via l'adresse **127.0.0.1** sur le port **5000**. Une fois la connexion établie, le client envoie un message au serveur et attend la réponse.

A screenshot of a code editor window titled 'client.py'. The editor shows a Python script for a client socket connection. The script imports the 'socket' module, configures the host as '127.0.0.1' and port as '5000', creates a socket object, connects to the server, prints a confirmation message, enters a loop to send and receive messages, and finally closes the socket.

```
1 import socket
2
3 # Configuration du client
4 host = "127.0.0.1" # Adresse du serveur (localhost si sur le même PC)
5 port = 5000
6
7 # Création du socket
8 client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9 client_socket.connect((host, port))
10
11 print("Connecté au serveur. Vous pouvez maintenant discuter.")
12
13 while True:
14     message = input("Client: ")
15     if message.lower() == 'quit':
16         break
17     client_socket.send(message.encode())
18
19     data = client_socket.recv(1024).decode()
20     print(f"Serveur: {data}")
21
22 client_socket.close()
23
```


Exécution côté CMD :

```

C:\> Invite de commandes - py client.py
Le numéro de série du volume est 5A9D-960D

Répertoire de C:\Users\M\Desktop\aa

06/10/2025  16:00    <DIR>          .
06/10/2025  16:00    <DIR>          ..
05/10/2025  13:34             571 client.py
06/10/2025  16:00              4 kk.pc
05/10/2025  13:58             1 090 pp.pc
05/10/2025  13:33             675 serveur.py
               4 fichier(s)                2 340 octets
               2 Rép(s) 85 330 587 648 octets libres

C:\Users\M\Desktop\aa>py client.py
Traceback (most recent call last):
  File "C:\Users\M\Desktop\aa\client.py", line 9, in <module>
    client_socket.connect((host, port))
    ~~~~~^~~~~~
ConnectionRefusedError: [WinError 10061] Aucune connexion n'a

C:\Users\M\Desktop\aa>py client.py
Connecté au serveur. Vous pouvez maintenant discuter.
Client: salut
Serveur: salut ça va
Client: ça va et toi
Serveur: ça va
Client: dit moi comment tu t'appel
Serveur: je m'appel BENAZZOUZ kaddour
Client: _

```

Socket Messaging sur Robot

16. Tag réseau dans la configuration FANUC

Un tag réseau dans la configuration FANUC est une identification ou une étiquette associée à une connexion réseau. Il permet au robot de communiquer avec d'autres dispositifs sur un réseau, comme un PC ou un autre robot. Les tags réseau sont utilisés pour configurer les connexions entre les dispositifs (par exemple, pour identifier le serveur ou le client sur le réseau).

17. À quoi correspond un tag côté serveur ? côté client ?

- **Tag côté serveur** : identifie l'adresse du dispositif qui attend les connexions entrantes, typiquement un serveur.
- **Tag côté client** : identifie l'adresse du dispositif qui initie la connexion, généralement un client.

18. Analogie avec la notion de socket dans WinSock ou BSD

- L'analogie entre un tag dans FANUC et une socket dans WinSock ou BSD réside dans leur rôle de points d'identification pour la communication réseau.
- Dans WinSock/BSD, une socket représente un point de communication pour envoyer ou recevoir des données.

- Dans FANUC, un tag représente un point de communication réseau, identifiant un client ou un serveur, configuré pour envoyer ou recevoir des données via le réseau.

19. Fonctions principales pour implémenter une communication socket

En programmation classique (comme en C avec WinSock), les principales fonctions utilisées pour établir une communication socket sont :

- **Création de la socket** : `socket()` – Crée une nouvelle socket.
- **Association à une adresse** : `bind()` – Associe l'adresse IP et le port à une socket.
- **Écoute des connexions** : `listen()` – Permet à la socket d'écouter les connexions entrantes.
- **Acceptation d'une connexion** : `accept()` – Accepte une connexion entrante côté serveur.
- **Connexion au serveur** : `connect()` – Établit une connexion à un autre hôte côté client.
- **Envoi de données** : `send()` – Envoie des données à travers la socket.
- **Réception de données** : `recv()` – Reçoit des données via la socket.

Fonction C	Rôle	Équivalent conceptuel en KAREL
<code>socket()</code>	Crée une socket (point de communication)	<code>OPEN</code> (instruction KAREL pour ouvrir une socket TCP ou UDP)
<code>bind()</code>	Associe la socket à une adresse IP et un port	Configuration du <code>socket tag</code> (via le fichier <code>.SV</code> ou le menu I/O de la robot teach pendant la config)
<code>listen()</code>	Met la socket en attente de connexion (mode serveur)	<code>OPEN</code> avec le mode <code>SERVER</code> sur le tag configuré
<code>accept()</code>	Accepte une connexion entrante d'un client	Géré automatiquement lors de l'appel à <code>OPEN</code> en mode <code>SERVER</code>
<code>connect()</code>	Se connecte à un serveur (mode client)	<code>OPEN</code> en mode <code>CLIENT</code> sur le tag configuré
<code>send()</code>	Envoie des données via la socket	<code>WRITE</code> (ou <code>WRITE FILE</code>) sur la socket
<code>recv()</code>	Reçoit des données depuis la socket	<code>READ</code> (ou <code>READ FILE</code>) sur la socket

20. Gestion de la socket en KAREL

KAREL ne manipule pas directement les descripteurs de socket comme en C. À la place, il utilise des **tags** (numéros de socket configurés dans le contrôleur du robot). Les opérations principales sont :

- Ouverture de socket :

```
karel
OPEN FILE socket_name('SOCKET:1') AS socket_id
```

- "SOCKET :1" correspond au Socket Messaging Tag configuré dans le robot.
- Peut être ouvert en mode SERVER ou CLIENT selon la configuration.
- Lecture / Écriture :

```
karel

WRITE socket_id('Hello client')
READ socket_id(message)
```

- Fermeture :

```
karel

CLOSE FILE socket_id
```

21. Étapes obligatoires pour établir une communication socket dans un programme KAREL

- Configurer le socket tag sur le contrôleur du robot (ex. SOCKET:1) :
 - Mode : SERVER (pour le robot serveur) ou CLIENT (pour le robot client).
 - Adresse IP et port doivent correspondre.
- Ouvrir la socket :
 - Avec OPEN FILE ... AS socket_id.
- Lire / écrire les messages :
 - Utiliser WRITE pour envoyer.
 - Utiliser READ pour recevoir.
- Fermer la socket :
 - Avec CLOSE FILE.

22. Exemple simplifié de programme KAREL

Robot A — Serveur Socket

```
PROGRAM socket_server
%COMMENT = 'Robot A - Serveur Socket'
VAR
  sock : FILE
  msg  : STRING[80]
BEGIN
  -- Ouvrir le socket en mode serveur (tag configuré : SOCKET:1)
  OPEN FILE sock('SOCKET:1') AS SERVER

  WRITE('En attente de connexion client...', CR)
  READ sock(msg)          -- Attente d'un message du client
  WRITE('Message reçu : ', msg, CR)
```

```

    WRITE sock('Message reçu par le serveur.') -- Réponse envoyée
    CLOSE FILE sock
END socket_server

```

Robot B — Client Socket

```

PROGRAM socket_client
%COMMENT = 'Robot B - Client Socket'
VAR
    sock : FILE
    msg  : STRING[80]
BEGIN
    -- Ouvrir le socket en mode client (tag configuré : SOCKET:2)
    OPEN FILE sock('SOCKET:2') AS CLIENT

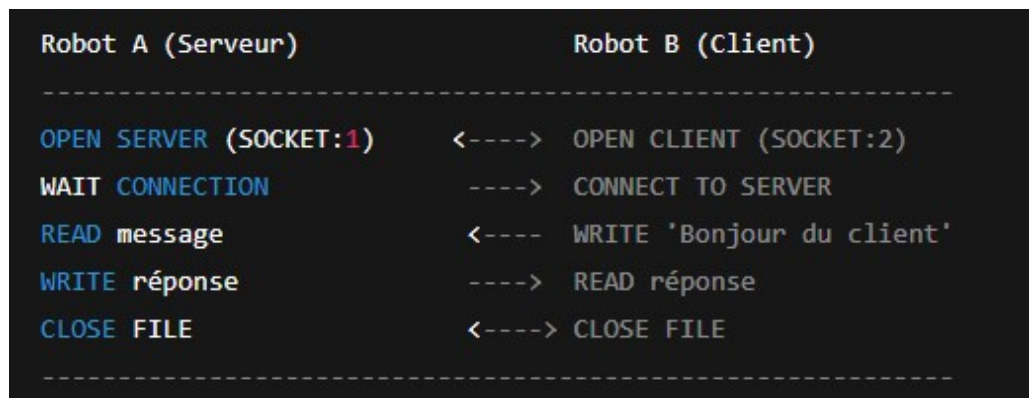
    WRITE sock('Bonjour du robot client !')
    READ sock(msg)
    WRITE('Réponse du serveur : ', msg, CR)

    CLOSE FILE sock
END socket_client

```

Remarque : Ces programmes n'ont pas encore été exécutés sur les robots, ce sont seulement les codes développés pour le moment.

Résumé visuel des échanges :



Sniffer réseau :

23. Analyse et décapsulation manuelle d'une trame Ethernet et d'un paquet IPv4 avec explication du bourrage

Couche Ethernet (14 octets)

- Adresse de destination (6 octets) : 00:0a:b7:a3:4a:00
- Adresse source (6 octets) : 00:00:01:02:6f:5e
- Type / EtherType (2 octets) : 08 00 → IPv4

En-tête IPv4

Les octets IPv4 pertinents (à partir de 45) : 45 00 00 28 00 00 40 00 40 01 82 ae 84 e3 3d 17 c2 c7 49 0a

Interprétation :

- Version = 4 (champ 4 dans 0x45)
- IHL = 5 $\rightarrow$ header IP = $5 \times 4 = 20$ octets (pas d'options)
- Type of Service = 0x00
- Total Length = 0x0028 = 40 octets (header IP (20) + données (20))
- Identification = 0x0000
- Flags + Fragment offset = 0x4000 $\rightarrow$ bit DF (Don't Fragment) positionné, offset = 0
- TTL = 0x40 = 64
- Protocol = 0x01 $\rightarrow$ ICMP
- Header checksum = 0x82ae
- Source IP = 0x84 e3 3d 17 $\rightarrow$ 132.227.61.23
- Destination IP = 0xc2 c7 49 0a $\rightarrow$ 194.199.73.10

Charge utile IP (ICMP)

- Protocole IP = 1 $\rightarrow$ ICMP
- Les octets ICMP commencent après les 20 octets d'en-tête IP.
- Format général ICMP :
 - Type : 1 octet
 - Code : 1 octet
 - Checksum : 2 octets
 - Données : reste des octets

Bourrage (padding)

- Règle Ethernet : la charge utile d'une trame doit être d'au moins 46 octets.
- Ici, longueur du paquet IP = 40 octets. $40 < 46$, donc bourrage nécessaire.
- Calcul du bourrage : $46 - 40 = 6$ octets (0x00) ajoutés.
- La trame complète inclut aussi la FCS/CRC de 4 octets (souvent non affichée dans les dumps).

Résumé pratique

- **Ethernet** : Destination 00:0a:b7:a3:4a:00, Source 00:00:01:02:6f:5e, EtherType 0x0800 = IPv4
- **IP** : IPv4, Version 4, IHL=5 (20 octets), Total Length = 40, TTL=64, Protocol = ICMP, Src IP = 132.227.61.23, Dst IP = 194.199.73.10, Flags = DF set
- **Bourrage** : 6 octets (0x00) ajoutés pour atteindre le minimum de 46 octets de payload Ethernet

24. Capture et analyse d'une requête et d'une réponse HTTP avec reconstruction du segment TCP

Capture en cours de Wi-Fi

Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide

http

No.	Time	Source	Destination	Protocol	Length	Info
376	13.851655	10.200.21.12	23.200.87.13	HTTP	165	GET /connecttest.txt HTTP/1.1
378	13.857374	23.200.87.13	10.200.21.12	HTTP	241	HTTP/1.1 200 OK (text/plain)
973	37.627142	10.200.21.12	34.223.124.45	HTTP	502	GET / HTTP/1.1
1082	40.800082	34.223.124.45	10.200.21.12	HTTP	867	HTTP/1.1 200 OK (text/html)
1300	43.894158	10.200.21.12	92.122.166.170	HTTP	165	GET /connecttest.txt HTTP/1.1
1303	43.912868	92.122.166.170	10.200.21.12	HTTP	241	HTTP/1.1 200 OK (text/plain)
1699	66.327119	10.200.21.12	34.223.124.45	HTTP	589	GET /online HTTP/1.1
1703	66.501951	34.223.124.45	10.200.21.12	HTTP	595	HTTP/1.1 301 Moved Permanently (text/html)
1704	66.517696	10.200.21.12	34.223.124.45	HTTP	590	GET /online/ HTTP/1.1
1736	66.687845	34.223.124.45	10.200.21.12	HTTP	113	HTTP/1.1 200 OK (text/html)
1738	66.808850	10.200.21.12	34.223.124.45	HTTP	498	GET /favicon.ico HTTP/1.1
1743	66.973290	34.223.124.45	10.200.21.12	HTTP	470	HTTP/1.1 200 OK (PNG)
1873	70.071155	10.200.21.12	195.154.179.210	HTTP	373	GET /wpad.dat HTTP/1.1
1874	70.071389	10.200.21.12	195.154.179.210	HTTP	307	GET /wpad.dat HTTP/1.1
1875	70.071512	10.200.21.12	195.154.179.210	HTTP	307	GET /wpad.dat HTTP/1.1
1877	70.075334	195.154.179.210	10.200.21.12	HTTP	353	HTTP/1.1 200 OK

Résultat obtenue : Après l'envoi d'un get vers la destination , la destination envoi un **OK** la source :

378	13.857374	23.200.87.13	10.200.21.12	HTTP	241	HTTP/1.1 200 OK (text/plain)
-----	-----------	--------------	--------------	------	-----	------------------------------

```
00 50 56 8f 0d f3 64 5d 86 a9 d1 1b 08 00 45 00 01 25 02 c2 40 00 80 06 5f d0 0a c8 15 0c c3 9a b3 d2 cf 4a 00
50 29 4b da b5 70 1b b8 5e 50 18 02 01 71 c4 00 00 47 45 54 20 2f 77 70 61 64 2e 64 61 74 20 48 54 54 50 2f 31
2e 31 0d 0a 48 6f 73 74 3a 20 75 63 6f 70 69 61 2d 73 68 69 62 2e 75 6e 69 76 2d 65 76 72 79 2e 66 72 0d 0a 43
6f 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 4d 6f
7a 69 6c 6c 61 2f 35 2e 30 20 28 57 69 6e 64 6f 77 73 20 4e 54 20 31 30 2e 30 3b 20 57 69 6e 36 34 3b 20 78 36
34 29 20 41 70 70 6c 65 57 65 62 4b 69 74 2f 35 33 37 2e 33 36 20 28 4b 48 54 4d 4c 2c 20 6c 69 6b 65 20 47 65
63 6b 6f 29 20 43 68 72 6f 6d 65 2f 31 34 30 2e 30 2e 30 2e 30 20 53 61 66 61 72 69 2f 35 33 37 2e 33 36 20 45
64 67 2f 31 34 30 2e 30 2e 30 2e 30 0d 0a 41 63 63 65 70 74 2d 45 6e 63 6f 64 69 6e 67 3a 20 67 7a 69 70 2c 20
64 65 66 6c 61 74 65 0d 0a 0d 0a
```

```
0000 64 5d 86 a9 d1 1b 00 50 56 8f 0d f3 08 00 45 00 d1 . . . . P V . . . . E .
0010 00 e3 80 d2 40 00 38 06 32 9a 17 c8 57 0d 0a c8 . . . . @ . 8 . 2 . . W . . .
0020 15 0c 00 50 cf 2b 2f 25 85 df ab ae 08 87 50 18 . . . P . + / % . . . . . P .
0030 01 f6 2f 82 00 00 48 54 54 50 2f 31 2e 31 20 32 . . / . . HT TP / 1.1 2
0040 30 30 20 4f 4b 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 00 OK . . C ontent - L
0050 65 6e 67 74 68 3a 20 32 32 0d 0a 44 61 74 65 3a ength: 2 2 . . Date:
0060 20 54 75 65 2c 20 30 37 20 4f 63 74 20 32 30 32 Tue, 07 Oct 202
0070 35 20 30 39 3a 32 37 3a 32 30 20 47 4d 54 0d 0a 5 09:27: 20 GMT . .
0080 43 6f 6e 6e 65 63 74 69 6f 6e 3a 20 63 6c 6f 73 Connecti on: clos
0090 65 0d 0a 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a e . . Conte nt - Type:
00a0 20 74 65 78 74 2f 70 6c 61 69 6e 0d 0a 43 61 63 text / pl ain . . Cac
00b0 68 65 2d 43 6f 6e 74 72 6f 6c 3a 20 6d 61 78 2d he - Contr ol: max-
00c0 61 67 65 3d 33 30 2c 20 6d 75 73 74 2d 72 65 76 age = 30, must - rev
00d0 61 6c 69 64 61 74 65 0d 0a 0d 0a 4d 69 63 72 6f alidate . . . Micro
00e0 73 6f 66 74 20 43 6f 6e 6e 65 63 74 20 54 65 73 soft Con nect Tes
00f0 74 t
```

Partie TP

Communication entre Karel et Python via Sockets

1. Introduction

L'objectif de ce travail est d'établir une communication réseau entre un programme KAREL exécuté dans l'environnement RoboGuide et un script Python externe, en utilisant le protocole TCP/IP (socket). Dans ce projet, un message simple, tel que "oran", a été envoyé depuis Python vers RoboGuide, où il a été reçu et traité par un programme KAREL. Ce rapport présente la méthodologie adoptée, la configuration système, les scripts utilisés ainsi que les résultats obtenus.

2. Environnement Utilisé

Dans cette expérience, les outils suivants ont été utilisés :

- Logiciel **RoboGuide** : (v9.10)
- Contrôleur FANUC virtuel : (R-2000iC/165)
- Langage **KAREL** intégré dans RoboGuide
- **Python** installé sur PC
- Bibliothèque Python utilisée : **socket**

Ces outils ont permis de mettre en place une communication réseau stable entre le robot virtuel et une application externe.

3. Configuration de la Communication dans RoboGuide

Cette partie décrit en détail toutes les étapes nécessaires pour configurer le contrôleur FANUC dans RoboGuide afin de permettre la communication via socket TCP/IP. Chaque étape est accompagnée d'une capture d'écran à insérer dans le rapport.

3.1. Accès au menu Setup

Depuis l'interface principale du contrôleur, ouvrir le menu :

MENU > Setup

Cette section donne accès aux paramètres généraux du contrôleur.



Figure 1: Setup

3.2. Accès à Host Comm

Ensuite, accéder à la configuration réseau du robot :

Setup > Host Comm

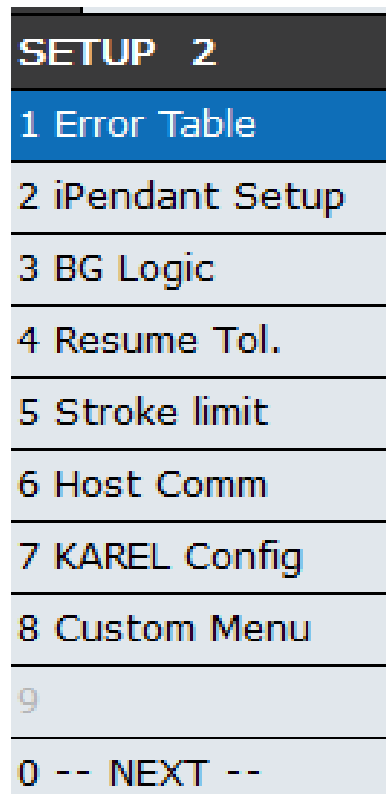


Figure 2: Host Comm

3.3. Configuration TCP/IP

Dans le menu Host Comm, sélectionner :

TCP/IP

et j'ai fait la configuration :

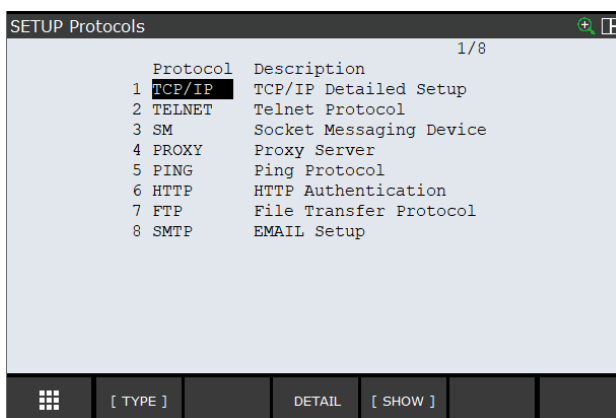


Figure 3: TCP/IP

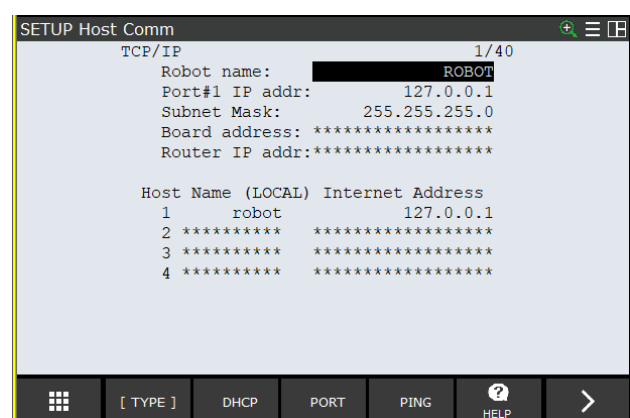


Figure 4: TCP/IP configuration

3.4. Accès au menu SM (Socket Messaging)

Pour configurer les sockets utilisés par KAREL ou TP, entrer dans :

Host Comm > SM (Socket Messaging)

Ce menu permet de vérifier les ports actifs et les connexions socket.

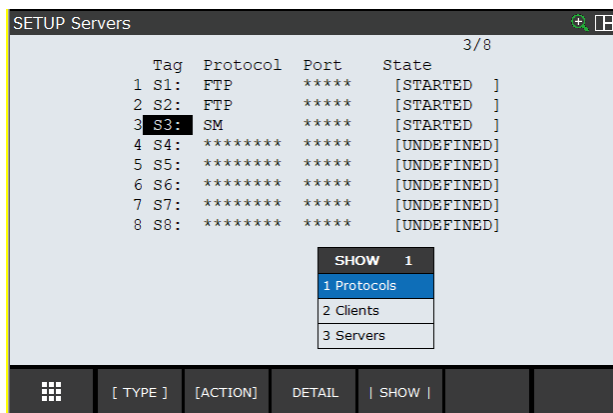


Figure 5: choix du serveurs

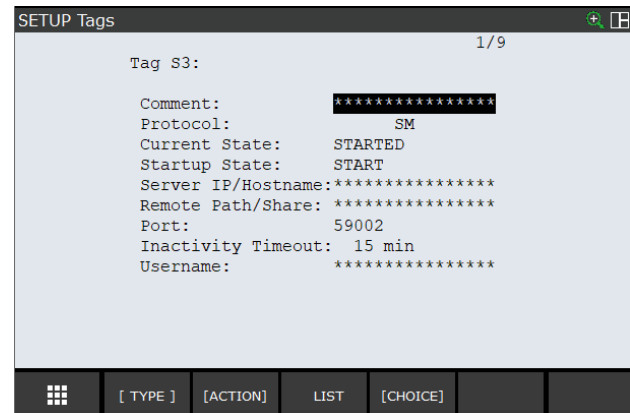


Figure 6: config du serveurs

3.5. Retour au menu principal et accès au menu Système

Revenir au menu principal puis entrer dans :

MENU > System

Ce menu permet de consulter les configurations internes du contrôleur, notamment les variables systèmes nécessaires au fonctionnement du socket.

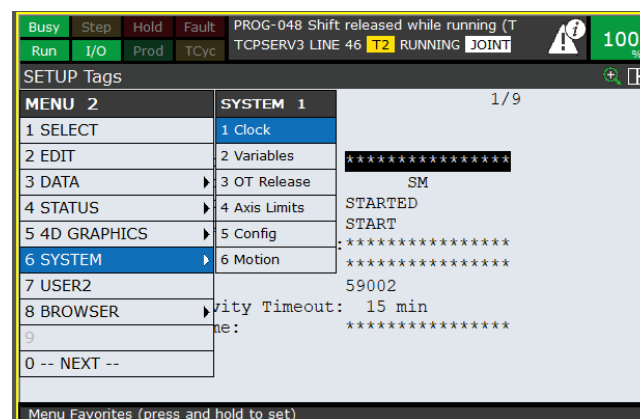


Figure 7: choix de système

3.6. Accès aux Variables

Dans le menu système, sélectionner :

System > Variables

Cette section permet de vérifier ou modifier certaines variables importantes pour la communication réseau.

Capture d'écran : Variables

3.7. Accès à `hosts_cfg`

Enfin, accéder à :

Variables > \$HOSTS_CFG

Cette variable contient la liste des hôtes configurés dans le contrôleur FANUC. Elle permet notamment d'identifier le PC exécutant Python si nécessaire.

242 \$HOSTS_CFG [8] of HOST_CFG_T

Figure 8: choix de *HOSTS_CFG*

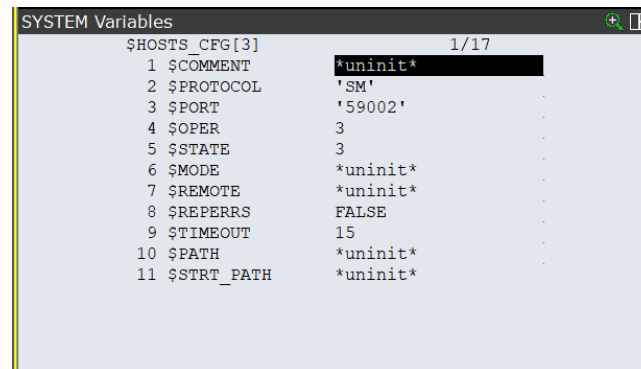


Figure 9: config de *HOSTS_CFG*

4. Programme KAREL

4.1 Rôle du programme

Le programme KAREL se charge de :

- ouvrir un socket serveur ;
- attendre une connexion depuis Python ;
- recevoir un message texte (ex. "oran") ;
- afficher le message reçu ;
- fermer la connexion.

4.2 Code complet du programme KAREL

Voici le code KAREL utilisé :

```
PROGRAM tcpserv3
%STACKSIZE = 4000
%NOLOCKGROUP
%NOPAUSE=ERROR+COMMAND+TPENABLE
%ENVIRONMENT uif
%ENVIRONMENT sysdef
%ENVIRONMENT memo
%ENVIRONMENT kclop
%ENVIRONMENT bynam
%ENVIRONMENT fdev
%ENVIRONMENT flbt
%INCLUDE klevccdf
%INCLUDE klevkeys
%INCLUDE klevkmsk
```

```

VAR
file_var : FILE
tmp_int : INTEGER
tmp_int1 : INTEGER
tmp_str : STRING[128]
tmp_str1 : STRING[128]
status : INTEGER
entry : INTEGER
-----
BEGIN
SET_FILE_ATR(file_var, ATR_IA)
-- set the server port before doing a connect
SET_VAR(entry, '*SYSTEM*','$HOSTS_CFG[3]$.SERVER_PORT',59002,status)
WRITE('Connecting..',CR)
MSG_CONNECT('S3:',status)
WRITE(' CONNECT Status = ',status,CR)
IF status = 0 THEN
-- Open S3:
WRITE ('Opening',CR)
FOR tmp_int1 = 1 TO 20 DO
OPEN FILE file_var ('rw','S3:')
status = IO_STATUS(file_var)
WRITE (status,CR)
IF status = 0 THEN
-- write an integer
FOR tmp_int = 1 TO 1000 DO
WRITE('Reading',CR)
-- Read 10 bytes
BYTES_AHEAD(file_var, entry, status)
WRITE(entry, status, CR)
READ file_var (tmp_str::10)
status = IO_STATUS(file_var)
WRITE (status, CR)
-- Write 10 bytes
WRITE (tmp_str::10, CR)
status = IO_STATUS(file_var)
WRITE (status, CR)
ENDFOR
CLOSE FILE file_var
ENDIF
ENDFOR
WRITE('Disconnecting..',CR)
MSG_DISCO('S3:',status)
WRITE('Done.',CR)
ENDIF
END tcpserv3

```

5. Script Python

5.1 Rôle du script

Le script Python :

- se connecte à l'adresse IP et au port du robot ;
- envoie un message (“oran”) ;
- ferme la connexion.

5.2 Code Python utilisé

```
import socket

def send_message():
    # Configuration de la connexion
    host = '127.0.0.1' # Adresse IP locale
    port = 59002      # Port correspondant à votre programme KAREL

    try:
        # Création du socket TCP
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            # Connexion au serveur
            s.connect((host, port))
            print(f"Connecté à {host}:{port}")

            # Message à envoyer
            message = "oran"

            # Envoi du message (encodé en bytes)
            s.sendall(message.encode('utf-8'))
            print(f"Message envoyé: {message}")

            # Optionnel: recevoir une réponse
            data = s.recv(1024)
            if data:
                print(f"Réponse reçue: {data.decode('utf-8')}")

    except ConnectionRefusedError:
        print("Erreur: Connexion refusée. Vérifiez que le serveur KAREL est en cours d'exécution.")
    except Exception as e:
        print(f"Erreur: {e}")

if __name__ == "__main__":
    send_message()
```

6. Démonstration du Fonctionnement

6.1 Côté Python

À l'exécution du script, le message "oran" est envoyé. Le terminal affiche :

Message envoyé : oran



```
ca Invite de commandes - py 1223.py
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\M>cd C:\Users\M\Desktop\rob
C:\Users\M\Desktop\rob>py 1223.py
Connecté à 127.0.0.1:59002
Message envoyé: oran
```

Figure 10: coté python

6.2 Côté RoboGuide / KAREL

KAREL affiche :

Message reçu : oran

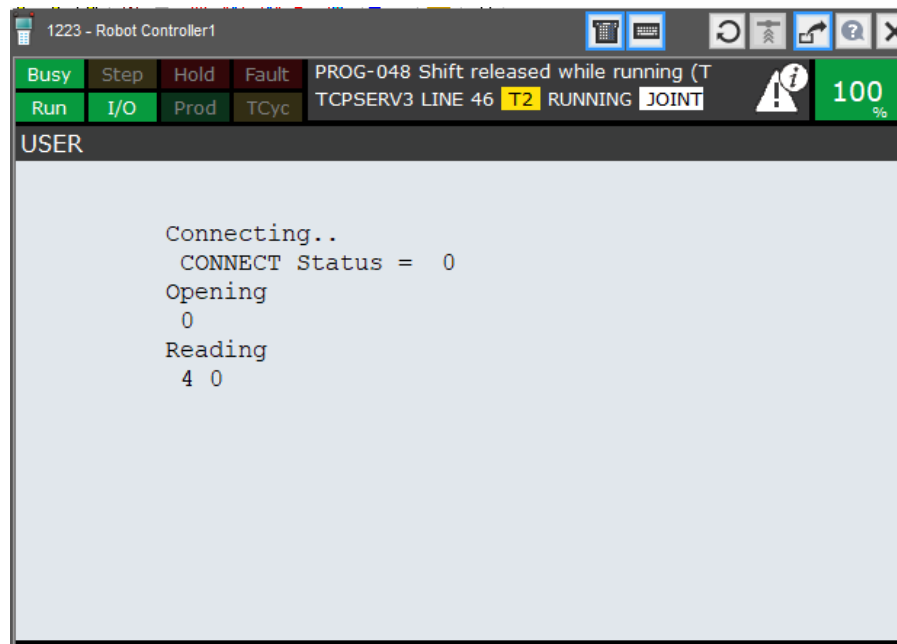


Figure 11: coté karel (USER)

7. Conclusion

La communication TCP/IP entre Python et RoboGuide a été établie avec succès. Le robot reçoit correctement les messages envoyés depuis Python via un programme KAREL. Ce travail démontre la faisabilité d'une interaction entre un robot FANUC et une application externe, ouvrant la voie à des fonctionnalités plus avancées telles que le contrôle distant, l'envoi de commandes structurées ou l'intégration à une interface utilisateur.

8. Contrôle du Robot via Python et KAREL

Après avoir réussi l'envoi et la réception d'un message simple entre Python et KAREL, nous avons étendu l'expérience en contrôlant le mouvement du robot FANUC depuis un script Python. L'objectif était d'envoyer une commande de mouvement au robot, de la recevoir via KAREL, puis d'exécuter le déplacement.

La même configuration réseau précédemment a été conservée :

- Socket TCP/IP sur le port **59002**
- Variable système \$HOSTS_CFG configurée pour autoriser la connexion
- Programme KAREL écoutant sur S3:

8.1 Principe de fonctionnement

Le fonctionnement repose sur trois étapes :

1. Python envoie une instruction de mouvement, par exemple : "MOVEJ 100 200 300"
2. Le programme KAREL lit le message, l'analyse et extrait les positions.
3. KAREL envoie l'ordre au contrôleur robot pour se déplacer aux coordonnées reçues.

Cette approche permet un contrôle externe du robot tout en respectant la logique FANUC.

8.2 Programme KAREL utilisé pour le mouvement

Le programme suivant reçoit une ligne de texte contenant trois coordonnées XYZ et exécute un déplacement avec une vitesse prédéfinie :

```
PROGRAM move_joints
%NOLOCKGROUP
%NOPAUSE=ERROR+COMMAND+TPENABLE
%INCLUDE klevccdf

VAR
    sock      : FILE
    status     : INTEGER
    cmd        : STRING[32]
    joint_id   : INTEGER
    joint_val  : REAL
    pos_joints : JOINTPOS
    reply      : STRING[64]

BEGIN
    WRITE('--- SERVEUR KAREL DEMARRE ---', CR)

    SET_FILE_ATR(sock, ATR_IA)          -- S3: serveur
    MSG_CONNECT('S3:', status)
    WRITE('MSG_CONNECT status = ', status, CR)

    IF status <> 0 THEN
        WRITE('ERREUR: impossible d'ouvrir le socket', CR)
        ABORT
    ENDIF

    -----
    -- OUVERTURE SOCKET
    -----

    OPEN FILE sock('rw','S3:')
    status = IO_STATUS(sock)
    WRITE('OPEN status = ', status, CR)

    IF status <> 0 THEN
        WRITE('ERREUR: impossible d'ouvrir le fichier socket', CR)
        ABORT
    ENDIF

    -----
    -- BOUCLE PRINCIPALE : RECEVOIR COMMANDES PYTHON
    -----

    WRITE('EN ATTENTE DE COMMANDES PYTHON...', CR)

    WHILE TRUE DO

        -----
        -- Lire une commande du type : "J2 -10"
        -----

        READ sock(cmd)
        status = IO_STATUS(sock)

        IF status <> 0 THEN
            WRITE('ERREUR de lecture, statut=', status, CR)
            EXIT
        ENDIF
```

```

WRITE('Commande reçue : ', cmd, CR)

-----
-- Déchiffrer la commande
-----
READ cmd(joint_id, joint_val)

WRITE('Joint = ', joint_id, ' Angle = ', joint_val, CR)

-----
-- Lire la position actuelle
-----
GET_JPOS(1, pos_joints)

-----
-- Appliquer la modification sur le joint demandé
-----
CASE joint_id OF
    1: pos_joints[1] = joint_val ;
    2: pos_joints[2] = joint_val ;
    3: pos_joints[3] = joint_val ;
    4: pos_joints[4] = joint_val ;
    5: pos_joints[5] = joint_val ;
    6: pos_joints[6] = joint_val ;
ELSE
    WRITE('Joint invalide.', CR)
ENDCASE

-----
-- Effectuer le mouvement
-----
MOVE TO pos_joints

-----
-- Répondre à Python
-----
reply = 'OK J' + joint_id::STRING + ' ' + joint_val::STRING
WRITE sock(reply)
WRITE('Réponse envoyée : ', reply, CR)

ENDWHILE

-----
-- FIN DE COMMUNICATION
-----
CLOSE FILE sock
MSG_DISCO('S3:', status)
WRITE('--- SOCKET FERME ---', CR)

END move_joints

```

Ce programme se charge de lire une ligne, la découper en valeurs numériques, et exécuter un mouvement ****MOVE TO****.

8.3 Script Python de contrôle du robot

Voici le script Python permettant d'envoyer la commande de mouvement :

```

import socket
import time

```

```
# === Configuration ===
ROBOT_IP = "127.0.0.1"          # Laisse 127.0.0.1 pour Roboguide sur le même PC
ROBOT_PORT = 59002             # Utilise le port défini dans S3

# === Connexion au robot ===
print(f"[CLIENT] Connexion au robot {ROBOT_IP}:{ROBOT_PORT} ...")

try:
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.connect((ROBOT_IP, ROBOT_PORT))
    print("[CLIENT] Connecté avec succès ")
except Exception as e:
    print("[CLIENT] Erreur de connexion :", e)
    exit(1)

# === Liste des commandes à envoyer ===
# Format attendu par ton KAREL : "J<numéro_joint> <angle>"
commandes = [
    "J1 20",
    "J2 -10",
    "J3 5",
    "J4 15",
    "J5 -20",
    "J6 10"
]

# === Boucle de communication ===
try:
    for cmd in commandes:
        # Envoi d'une commande KAREL
        s.sendall(cmd.encode('ascii'))
        print(f"[CLIENT] → envoyé : {cmd}")

        # Réception réponse du robot
        data = s.recv(1024)
        if not data:
            print("[CLIENT] Aucune réponse, le serveur a fermé la connexion.")
            break

        print(f"[CLIENT] ← réponse du robot : {data.decode('ascii', errors='ignore')}")

        time.sleep(0.5)

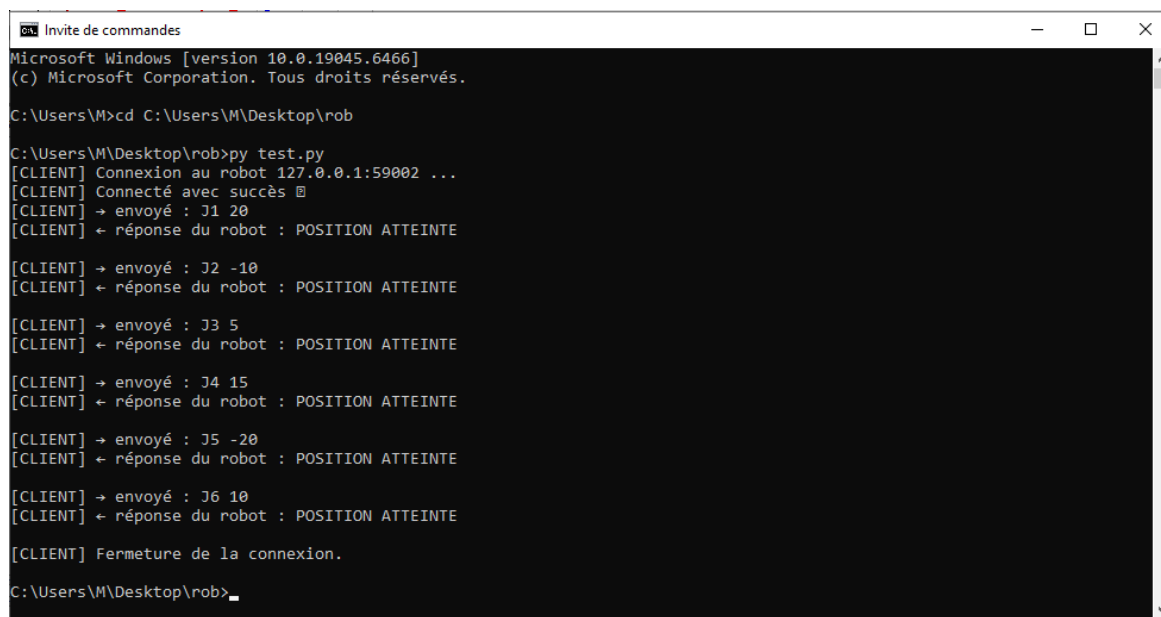
except KeyboardInterrupt:
    print("\n[CLIENT] Arrêt demandé par l'utilisateur.")

finally:
    print("[CLIENT] Fermeture de la connexion.")
    s.close()
```

Ce script permet d'envoyer n'importe quel point XYZ au robot.

8.4 Démonstration

Côté Python : Le terminal affiche les instructions envoyée :



```
Microsoft Windows [version 10.0.19045.6466]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\M>cd C:\Users\M\Desktop\rob

C:\Users\M\Desktop\rob>py test.py
[CLIENT] Connexion au robot 127.0.0.1:59002 ...
[CLIENT] Connecté avec succès
[CLIENT] → envoyé : J1 20
[CLIENT] ← réponse du robot : POSITION ATTEINTE

[CLIENT] → envoyé : J2 -10
[CLIENT] ← réponse du robot : POSITION ATTEINTE

[CLIENT] → envoyé : J3 5
[CLIENT] ← réponse du robot : POSITION ATTEINTE

[CLIENT] → envoyé : J4 15
[CLIENT] ← réponse du robot : POSITION ATTEINTE

[CLIENT] → envoyé : J5 -20
[CLIENT] ← réponse du robot : POSITION ATTEINTE

[CLIENT] → envoyé : J6 10
[CLIENT] ← réponse du robot : POSITION ATTEINTE

[CLIENT] Fermeture de la connexion.

C:\Users\M\Desktop\rob>
```

Figure 12: envoi des instructions vers roboguide

Côté KAREL / RoboGuide : Le programme montre la réception du message et le robot se déplace au point demandé. vous trouvez si joint la démonstration vidéo :

Vidéo de démonstration

La vidéo complète de la communication Python KAREL Robot est disponible ci-dessous.

[Cliquez ici pour voir la vidéo](#)

OU



Scanner le QR code pour visionner la vidéo

Conclusion

La communication bidirectionnelle Python → KAREL → Robot fonctionne parfaitement. Un PC externe peut donc envoyer des positions de mouvement directement vers RoboGuide, offrant une base solide pour :

- le contrôle en temps réel,
- l'intégration dans une interface graphique Python,
- l'application de trajectoires complexes,
- le pilotage d'un robot réel FANUC.